Invocación Remota

Introducción

Existen 10 tipos de personas:
Las que saben binario y las que no

Invocación Remota

Agenda

Modelo Cliente Servidor

¿Qué es la invocación remota?

Dificultades y consideraciones

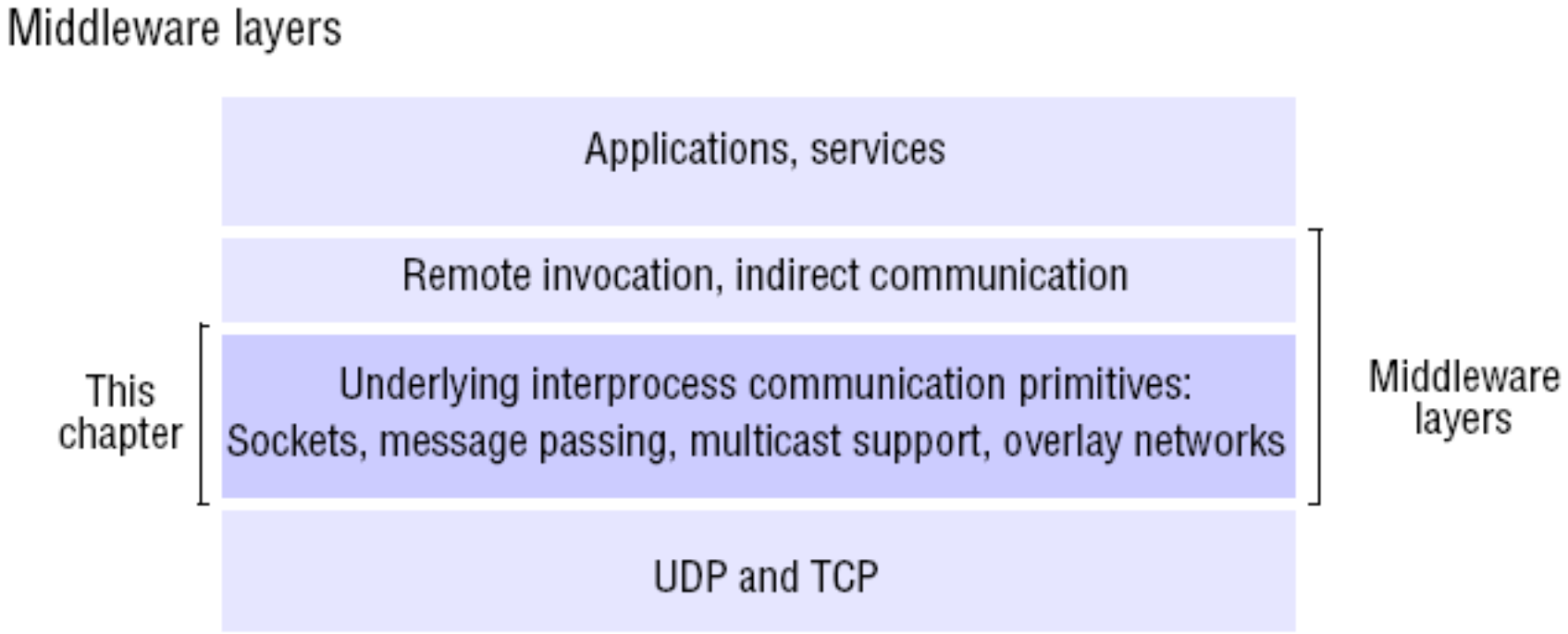
RPC

Java RMI

CORBA

Modelo cliente-servidor

Arquitectura del Middleware



Modelo de Interacción de software

- Socket API: todo lo necesario se obtiene del sistema operativo para acceder a la red
- Socket = distintos canales de comunicación de extremo a extremo interfaz de lectura/escritura
- Protocolos basados en texto orientados a líneas comunes

– No es eficiente pero fácil de depurar y usar

Ejemplo de interacción HTTP

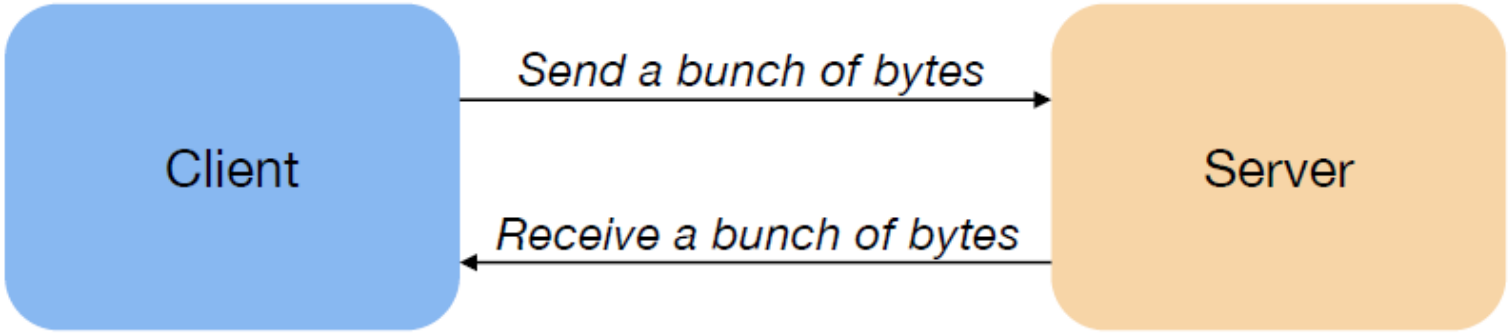
```
$ telnet www.google.com 80
Trying 172.217.12.196...
Connected to www.google.com.
Escape character is '^]'.
GET /index.html HTTP/1.1
HOST: www.google.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
User-Agent: Mozilla/4.0

HTTP/1.1 200 OK
Date: Sun, 29 Jan 2023 22:58:25 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
...
Transfer-Encoding: chunked

5584
<!doctype html><html itemscope=""
itemtype="http://schema.org/WebPage"
lang="en"><head>
      …
```

First part of the response – HTTP headers

Second part of the response – HTTP content

Problemas con los sockets

- La interface sockets fuerza a mecanismos de lectura/escritura



- La programación es a menudo más fácil con una interfaz funcional
- Para que la computación distribuida se parezca más a la computación centralizada, la E/S (lectura/escritura) no es el camino a seguir.

Middleware layer

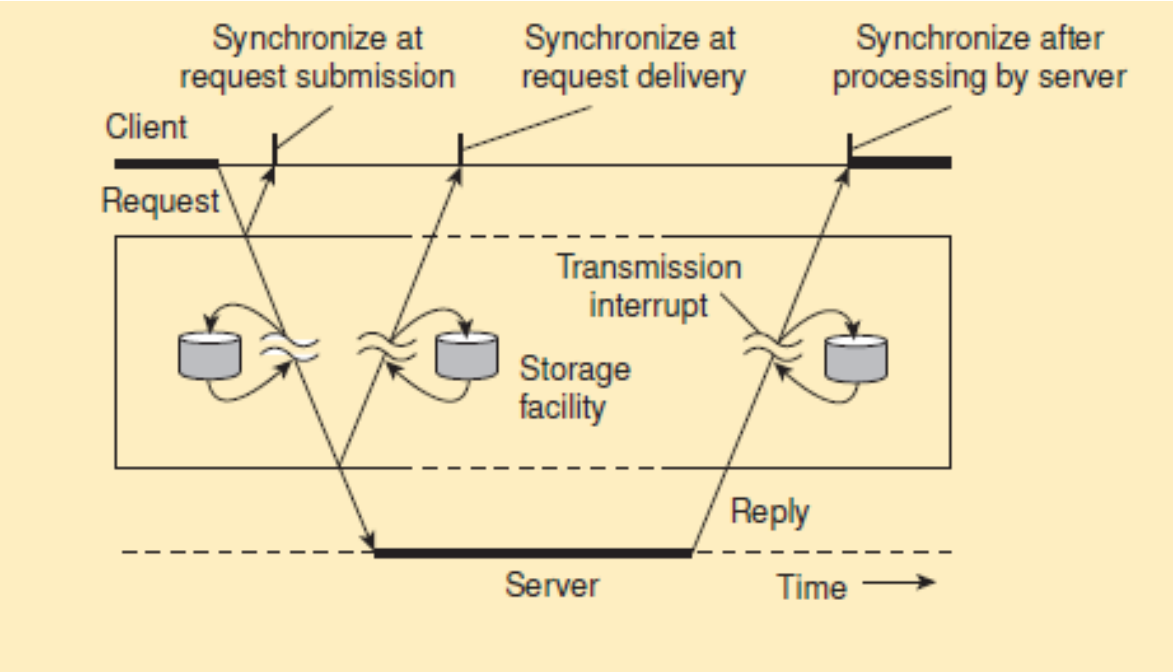
Middleware is invented to provide common services and protocols that can be used by many different applications

- A rich set of communication protocols

- (Un)marshaling of data, necessary for integrated systems

- Naming protocols, to allow easy sharing of resources
- Security protocols for secure communication
- Scaling mechanisms, such as for replication and caching

Types of communication



- Asynchronous versus synchronous
- Transient: Comm. server discards message when it cannot be delivered at the next server, or at the receiver.
- Persistent: A message is stored at a communication server as long as it takes to deliver it.

Invocación remota - RMI

- Se utiliza el termino RMI para referirnos a una invocación de un método remoto de forma genérica.
- Extiende el modelo de llamada convencional, al modelo de llamada a procedimiento remoto.
- Más reciente es extender el modelo POO para permitir la comunicación mediante RMI.

Styles of exchange protocols

Three protocols, that produce differing behaviours in the presence of communication failures are used for implementing various types of request behaviour. They were originally identified by Alfred Spector [1982]:

- the *request (R) protocol;*
- the *request-reply (RR) protocol;*
- the *request-reply-acknowledge reply (RRA) protocol.*

RPC exchange protocols

Name	*Messages sent by*		
	Client	*Server*	*Client*
R	*Request*		
RR	*Request*	*Reply*	
RRA	*Request*	*Reply*	*Acknowledge reply*

Semántica de invocación

- La semántica para invocaciones de métodos locales es exactamente una vez.
- Las semánticas de invocación remota se definen como:

- Semántica de invocación pudiera ser
- Semántica de invocación al menos una vez
- Semántica de invocación como máximo una vez

Semántica de invocación pudiera ser

- El que invoca no puede decir si un método se ha ejecutado una vez, o ninguna en absoluto.
- La semántica pudiera ser aparece cuando no se aplica ninguna medida de tolerancia a fallos.
- Los tipos de fallos pueden ser:

- fallos de omisión si se pierde la invocación o el mensaje con el resultado
- fallos por caída cuando el servidor que contiene el objeto remoto falla.

Si el mensaje con el resultado no se recibe tras un timeout y no hay reintentos, no hay certeza de si se ha ejecutado el método

Semántica al menos una vez

- El invocante recibe un resultado, en cuyo caso el invocante sabe que el método se evaluó al menos una vez, o informa una excepción.
- Pueden ocurrir los fallos:

  - fallo por caída cuando el servidor que contiene el objeto remoto falla
  - fallos arbitrarios. En casos donde el mensaje de invocación se retransmite, el objeto remoto puede recibirlo y ejecutar el método más de una vez, provocando que se guarden o devuelvan valores erróneos.

Debería utilizar operaciones idempotentes (se pueden ejecutar repetidamente)

Semántica como máximo una vez

- El invocante recibe bien un resultado, en cuyo caso el invocante sabe que el método se ejecutó exactamente una vez, o informa una excepción.
- La semántica puede obtenerse utilizando todas las medidas de tolerancia a fallos.
- *Tanto Java RMI, como CORBA observan la semántica de invocación como máximo una vez, pero CORBA permite emplear la semántica pudiera ser para los métodos que no devuelven resultados. Sun RPC proporciona la semántica al menos una vez.*

CODIFICACIÓN DE DATOS PARA EL INTERCAMBIO

- Cualquier mecanismo de invocación remota necesita pasar datos por la red en forma de secuencia de bytes.
- Hay que acordar un mecanismo de codificación (el emisor) y una interpretación de los datos recibidos (receptor).
- Hay que definir qué tipos de datos se van a utilizar (enteros, caracteres, reales), el orden en que se van a enviar estos datos, o juego de caracteres a utilizar.
- Hay que representar “en serie” un conjunto de datos que inicialmente no estaban pensados para este tipo de representación y además si es posible, debemos intentar compactarlo tanto como sea posible para disminuir el tráfico en la red y mejorar la velocidad de ejecución.

CODIFICACIÓN DE DATOS PARA EL INTERCAMBIO

- Hay varios formatos para reducir la complejidad y el coste de los datos:

- Enviar en el formato interno del emisor o del receptor. En cualquiera de los dos casos uno de los extremos no hace nada y el otro debería conocer como representar los datos para cualquier arquitectura.
- Enviar en forma canónica intermedia que debe ser conocida por cada computador.
- No conversión si las dos computadoras son similares (estamos haciendo dos conversiones innecesarias).
- En el formato del emisor incluyendo una indicación del formato, para que el receptor a la llegada de los mismos, pueda convertirlos.

Big-endian little-endian

- El **endian (**orden de representación de datos de varios bytes de longitud). Puede ser de dos tipos:

- **Little-endian**: el byte menos representativo se guarda en la dirección de memoria menor, son little-endian los Pentium, el sistema operativo Windows y distintos tipos de archivos; su ventaja es que las operaciones matemáticas son más fáciles de hacer porque el primer byte que se encuentran es el más pequeño y a partir de ahí en orden creciente, se van encontrando los siguientes bytes para operar.
- **Big-endian**: El byte más significativo se guarda en la dirección de memoria menor. Son de este tipo los procesadores Motorola, el superSparc de Sun, el sistema operativo Macintosh y sirve para saber rápidamente si un número es positivo o negativo, se guarda tal como se escribe.

Ejemplo de sintaxis

Representación del número 345 (1 0101 1001) en distintas arquitecturas:

Motorola Punta gruesa	0000 0001	0101 1001	
Intel Punta fina	0101 1001	0000 0001	
IBM system 370 Binario decimal empaquetado	0000 0011	0100 0101	
Mini/micro computadores ASCII (33h, 34h, 35h)	0011 0011	0011 0100	0011 0101
Mainframes EBCDIC (f3h, f4h, f5h)	1111 0011	1111 0100	1111 0101
	Menor dirección		Mayor dirección

Codificación

- Cada valor de datos necesita una **etiqueta** que aporta información para delimitar e identificar el valor, suele indicar el tipo de valor, la longitud o la arquitectura (endian).
- La **codificación** puede ser **textual o binaria**; con la primera todo se convierte a texto, es muy legible y se prefiere por su facilidad para analizar si algo falla, pero tiene el inconveniente que se usan muchos bits de información y la velocidad de ejecución y transmisión se resienten. Por el contrario, con la codificación binaria no se busca legibilidad, sino eficiencia.

Delimitación de datos

- Para **delimitar** los elementos a codificar hay varios opciones:
  - **Longitud fija:** es compacta e inflexible, el efecto 2000 se debió a este tipo de codificación. Solo es recomendable para datos que tengan siempre la misma longitud, pues en otro caso o desperdiciamos o corremos el riesgo de no poder expresar algún dato.
  - **Por longitud:** Se debe conocer la longitud con antelación y así al receptor lo primero que le llega es la medida exacta del dato, lo malo es que el emisor debe recorrer todo el dato para poder enviar la información del tamaño.
  - **Delimitada:** En lugar de calcular la longitud, se reserva un símbolo para separar datos (por ejemplo “;”).

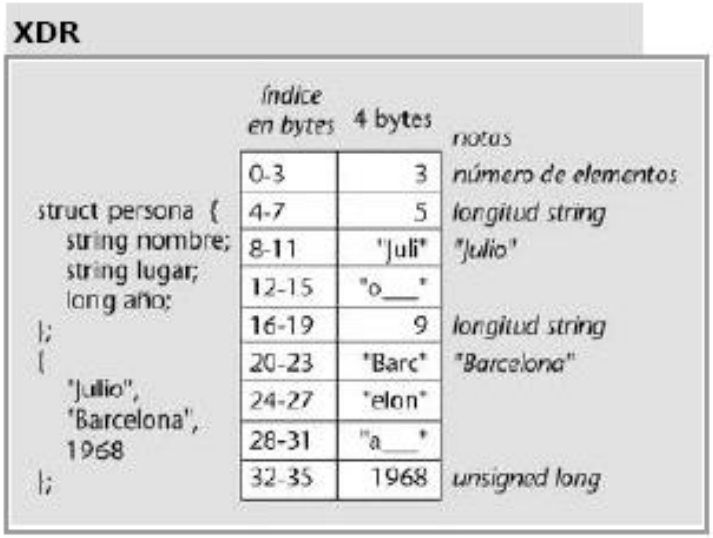
Marshalling - unmarshalling

- Se denomina marshalling (serialización o empaquetado) al proceso de transformar la representación de memoria de un objeto a un formato apto para su transmisión o almacenamiento.

- UnMarshalling es el proceso inverso, o sea generar lo valores primitivos desde la representación de datos externa y reconstruir las estructuras de datos

Formatos codificación de datos

- **External data representation (XDR)**
- Soporta todos los tipos del lenguaje C, define la forma canónica intermedia, no usa etiquetas y utiliza big-endian.



Formatos codificación de datos

XML

- Es el único sistema que representa datos en forma textual, no siendo una representación nada compacta, pero sí que muy informativa y fácil de entender.

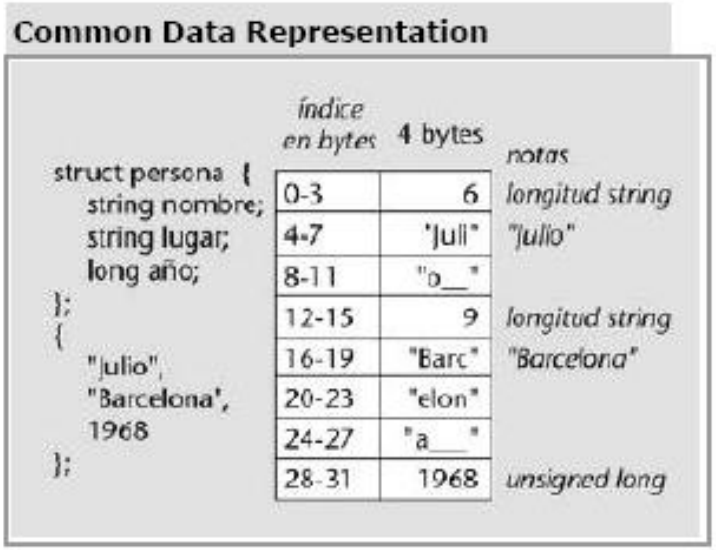
XML

```
<?xml version="1.0"?>
<methodCall>
<methodName>buscar</methodName>
<params>
<param>
<value>
<struct>
<member><name>nombre</name><value>Juan
</value></member>
<member><name>edad</name><value><i4>
42 </i4></value></member>
</struct></value></param>
</params></methodCall>
```

Formatos codificación de datos

- **Common data representation (CDR)**

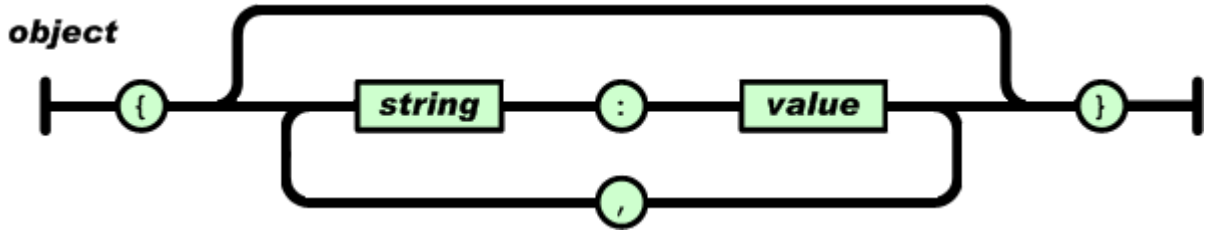
  Definido en Corba 2.0 permite representar tanto big como little-endian. Los valores se transmiten en el orden del emisor: se envía etiqueta de arquitectura, pero no se envían etiquetas de tipo pues es implícito: el emisor y el receptor ya lo han acordado previamente.



Formatos codificación de datos

- **JSON (**JavaScript Object Notation**)**

  Is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.



Formatos codificación de datos

- **Abstract syntax notation 1 (ASN.1)**

  Cada valor transmitido se codifica utilizando 4 campos: identificador, longitud, campo de datos, indicador de fin de contenido. Se usa con SNMP.
- **Serialización de objetos Java**

  Es específico para el lenguaje Java.
- **Protobuf**

  Protocol Buffers es un formato binario de serialización de datos, creado por Google. Almacena de forma eficiente y compacta datos estructurados. Es compatible con una amplia gama de lenguajes y es independiente de la plataforma.

Formatos codificación de datos

- La ventaja de XDR es su simplicidad, pues usa enteros de longitud fija y todos los datos ocupan múltiplos de 4 bytes, los más pequeños se redondean con bits a 0.
- CDR es más complejo que XDR y permite seleccionar entre big y little endian, lo que permite ahorrar conversiones.
- ASN.1 es más ineficiente en espacio y codificación; el mecanismo de empaquetado de tipos y longitudes hace que requiera más instrucciones para procesar los datos.
- XML ocupa mucho más espacio, el código para (de)codificar es más complejo y lento, pero su forma textual hace que sea muy fácil de depurar.
- JSON se está utilizando para transmitir datos entre aplicaciones Web.

Sistemas Distribuidos

JSON - JavaScript Object Notation

JSON

- JSON (JavaScript Object Notation) es un formato estándar abierto, que es ligero y basado en texto, diseñado explícitamente para el intercambio de datos legible por humanos.
- Es un formato de datos independiente del lenguaje. Es compatible con casi todo tipo de lenguaje, framework y biblioteca.
- JSON está diseñado para almacenar y organizar datos similares a XML, pero JSON es más pequeño, más rápido y más fácil de analizar, de escribir y leer.

JSON

- Fue especificado en el 2000 por Douglas Crockford, en 2013 se estandarizó como ECMA-404, y la RCF 8259 se publicó en 2017.
- JSON es un estándar abierto para el intercambio de datos en la web. Soporta estructuras de datos como objetos y matrices.
- Los datos se representan en pares clave-valor, y las llaves contienen objetos, donde se siguen dos puntos después de cada nombre. La coma se utiliza para separar los pares clave-valor. Los corchetes se utilizan para contener matrices, donde cada valor está separado por comas.

Ejemplo en JSON

```
{
   "students": [
      {
         "firstName": “Mauricio",
         "lastName": “Marinelli"
      },
      {
         "firstName": “Zumel",
         "lastName": “Candela"
      },
      {
         "firstName": “Espejo",
         "LastName": “Luciano"
      }
   ]  }
```

Tipos de datos JSON

Data Type	Description	Example
String	A string is always written in double-quotes. It may consist of numbers, alphanumeric and special characters.	"student", "name", "1234", "Ver_1"
Number	Number represents the numeric characters.	121, 899
Boolean	It can be either True or False.	true
Null	It is an empty value.	

Objetos JSON

Los objetos se refieren a diccionarios, que están encerrados entre corchetes, es decir, { }. Estos objetos se escriben en pares clave/valor, donde la clave tiene que ser una cadena y los valores tienen que ser un tipo de datos JSON válido, como cadena, número, objeto, booleano o nulo. Aquí la clave y los valores están separados por dos puntos, y una coma separa cada par clave/valor.

- Por ejemplo:

{"nombre" : "Jack", "employeeid" : 001, "presente" : false}

Matrices JSON

- Las matrices se pueden entender como una lista de objetos, que están encerrados entre corchetes [ ]. Un valor de matriz puede ser una cadena, número, objeto, matriz, booleano o nulo.
- Por ejemplo:

{ "PizzaName" : "Fiesta del País",

"Base" : "Cheese burst",

"Toppings" : ["Jalapeños", "Aceitunas negras", "Queso extra", "Salchichas", "Tomates cherry"],

"Picante" : "sí",

"Veg." : "sí"

}

¿Porqué usar JSON?

- El formato JSON es similar al código para crear objetos JavaScript. Debido a esto, un programa JavaScript puede convertir datos JSON en objetos JavaScript.
- Dado que el formato es solo texto, los datos JSON se pueden enviar fácilmente entre computadoras y ser utilizados por cualquier lenguaje de programación.
- JavaScript tiene una función incorporada para convertir cadenas JSON en objetos JavaScript: JSON.parse()
- JavaScript también tiene una función incorporada para convertir un objeto en una cadena JSON: JSON.stringify()

Consideraciones

- JSON significa JavaScript Object Notation.
- Es un formato de intercambio de datos estándar abierto.
- Es un formato solo de datos, no tiene métodos.
- Es ligero, autodescriptivo, fácil de leer y escribir.
- Es independiente del lenguaje.
- Admite estructuras de datos como matrices y objetos.

Consideraciones

- JSON requiere usar comillas dobles para las cadenas y los nombres de propiedades. Las comillas simples no son válidas.
- Una coma o dos puntos mal ubicados pueden producir que un archivo JSON no funcione. Se debe ser cuidadoso para validar cualquier dato que se quiera utilizar (aunque los JSON generados por computadora tienen menos probabilidades de tener errores). Es posible validar JSON utilizando una aplicación como JSONLint

Comparación JSON vs PB

	Protobuf	JSON
Desarrollador	Google	Douglas Crockford
Función	Formato de marcado para datos estructurados (almacenamiento y transmisión) y biblioteca	Formato de marcado para datos estructurados (almacenamiento y transmisión)
Formato binario	Sí	No
Estandarización	No	Sí
Legible para el ser humano	En parte	Sí
Comunidad/documentación	Comunidad pequeña; manuales en línea extensibles	Comunidad enorme, buena documentación oficial y diversos tutoriales en línea, etc.

Práctico

- Completar el primer ejemplo con diez alumnos del curso, agregando número de registro.
- Realizar un archivo JSON con datos de inventarios de artículos de informática (con al menos 5 campos) a su elección. Ej. Impresora, tipo, marca, etc. O algún software…
- Con los datos del censo 2022 elegir 10 campos y crear un JSON con 5 registros.

https://www.indec.gob.ar/ftp/cuadros/poblacion/cnphv2022_resultados_provisionales.pdf (páginas 71 a 78)

- Probarlos en JSONLint.
- Probar las funciones JSON.parse() y JSON.stringify() con 2 ejemplos de los ejercicios anteriores.
- Realizar los primero 2 ejercicios en Protobuf

Próximo Tema

RPC – Llamada a Procedimiento Remoto (Remote Procedure Call)